VLM Fine Tuning

지금까지 배운 것

- **1강**: VLM 개요 — VLM이 무엇이고 어떻게 동작하는지
- **2~4강**: 프롬프트 엔지니어링, Multimodal RAG, Video VLM — 전부 **추론 시점 (inference-time)** 기법
- 모델 가중치는 그대로 두고, **입력을 잘 설계**해서 원하는 답을 얻는 방법들이었음

이번 강의

- 프롬프트/RAG로도 안 되는 문제 → **모델 자체를 학습(fine-tuning)** 시켜야 함
- 실습은 **RunPod GPU 인스턴스**를 대여해 학습을 진행 (로컬 Ollama 환경과는 별개)

> 핵심 질문: **언제 fine-tuning이 필요하고, 무엇을(전체 vs 일부) 어떻게 학습시킬 것인가**

1. Fine-tuning이 필요한 경우

- 추론 시점 기법의 한계
- Fine-tuning을 고려해야 하는 신호
- 방법론 선택 기준

추론 시점 기법의 한계

문제 상황	프롬프트/RAG로 해결 가능?
모델이 모르는 최신/사내 지식이 필요	O — RAG로 문서 주입
답변 형식(JSON 등)을 지켜야 함	대체로 O — 프롬프트로 유도
특정 도메인 이미지의 **미묘한 패턴**을 인식해야 함 (예: 특정 불량 유형, 특정 화풍)	X — 말로 설명하기 어려움
항상 같은 톤·형식의 답변을 **높은 일관성**으로 내야 함	제한적 — 프롬프트만으로는 흔들림

> 프롬프트로 "설명"할 수 없는 패턴이거나, 매번 같은 지시를 반복해야 한다면 fine-tuning을 고려

Fine-tuning을 고려해야 하는 신호

- 프롬프트를 아무리 정교하게 짜도 **정확도가 일정 수준 이상 오르지 않음**
- 같은 형식의 데이터가 **충분히 많이 축적**되어 있음 (수백~수천 건 이상)
- 서비스가 **특정 도메인에 고정**되어 범용성보다 정확도·일관성이 중요
- 매 요청의 프롬프트 길이(토큰 비용)를 줄여야 하는 대량 트래픽 서비스

> 반대로 데이터가 적거나, 요구사항이 자주 바뀌는 초기 단계라면 RAG/프롬프트가 더 합리적

방법론 선택 기준

구분	핵심 질문	대표 선택지
목적	모델이 어떤 문제를 해결해야 하는가?	외부 지식 → RAG, 답변 행동 → Instruction Tuning
방식	제한된 GPU로 어떻게 학습할 것인가?	Full Fine-tuning, LoRA, QLoRA
대상	모델의 어느 부분을 학습할 것인가?	Projector, LLM 일부, Vision Adapter
평가	실제로 좋아졌는지 어떻게 확인할 것인가?	Held-out 평가셋, 전후 비교, 회귀 체크

2. VLM 학습 데이터 구조

- Instruction-following 데이터 포맷
- 이미지-텍스트 페어 구성
- 데이터 품질 체크리스트

Instruction-following 데이터 포맷

- LLM의 instruction tuning 데이터(`instruction` - `input` - `output`) 구조에 **이미지**가 추가된 형태
- 대화형(conversation) 포맷이 표준 — 여러 턴의 Q&A를 하나의 샘플로 구성 가능

```
{
  "image": "images/0001.jpg",
  "conversations": [
    {"from": "human", "value": "<image>\n이 제품 사진에서 불량 부위를 찾아줘."},
    {"from": "gpt", "value": "우측 상단 모서리에 스크래치가 있습니다."}
  ]
}
```

이미지-텍스트 페어 구성

- **캡션형**: 이미지 1장 + 설명 1개 — 가장 단순, VQA/캡셔닝 기초 학습에 사용
- **대화형**: 이미지 1장 + 여러 턴의 Q&A — 실제 서비스 시나리오에 가까움
- **비교/근거형**: 이미지 + 정답 + "왜 그런지"에 대한 근거(rationale) — 판단 근거를 함께 학습시켜 신뢰도 향상

유형	장점	단점
캡션형	제작 쉬움	실제 질의응답 패턴과 거리 있음
대화형	서비스 패턴과 유사	제작 비용 높음(멀티턴 라벨링)
근거형	설명 가능성 확보	라벨러의 근거 작성 부담 큼

데이터 품질 체크리스트

- **다양성**: 같은 유형의 이미지/질문만 반복되면 과적합(특정 패턴만 암기)
- **일관성**: 같은 상황에 다른 정답 포맷이 섞이면 학습이 흔들림
- **정답 검증**: 라벨링 오류가 소수만 섞여도 fine-tuning 품질에 큰 영향
- **클래스 불균형**: 특정 케이스(예: "이상없음")가 과도하게 많으면 편향 학습
- **평가셋 선분리**: 학습 전에 train/val/test를 먼저 나누고, 무엇을 성공으로 볼지 기준을 정해야 함

> 데이터 100건을 정제하는 것이, 1,000건을 대충 모으는 것보다 나은 경우가 많음

> 실습: 노트북 1(`1. VLM 학습 데이터 준비.ipynb`)에서 위 포맷과 체크리스트를 직접 코드로 확인

3. 학습 목적: Instruction Tuning

- 개념과 목적
- VLM 구조에서 무엇을 학습시키는가
- 학습 대상과 방식 조합

개념과 목적

- 사전학습(pretraining)된 VLM에 **"지시를 따르는 방식"**을 가르치는 후속 학습 단계
- 목표: 새로운 지식 주입이 아니라, **원하는 형식·태도로 답하도록 정렬(align)**
- Instruction Tuning은 **학습 목적/데이터 방식**이고, LoRA/QLoRA는 이를 **적은 자원으로 수행하는 학습 방식**
- Full fine-tuning(전체 파라미터 학습)은 VLM 규모에서는 비용·메모리 부담이 매우 큼

VLM 구조에서 무엇을 학습시키는가

VLM은 보통 **Vision Encoder** + **Projector(연결부)** + **LLM** 3부분으로 구성

구성요소	역할	Instruction Tuning에서
Vision Encoder	이미지를 임베딩으로 변환	보통 **freeze**(동결) — 이미 잘 학습되어 있음
Projector	이미지 임베딩을 LLM 입력 공간으로 매핑	**학습 대상** — 새 도메인 적응에 핵심
LLM(언어모델)	텍스트 생성	전체 또는 일부(LoRA)만 학습

학습 대상과 방식 조합

설계	대상	방식	비용	주 용도
Full Fine-tuning	전체	전체 가중치	매우 높음	대규모 전환
Projector + LLM	Projector + LLM	전체/부분 업데이트	높음	형식·스타일 큰 변경
Projector only	Projector	연결부만 업데이트	낮음	매핑 보정
LoRA/QLoRA 기반	LLM 일부 (+Projector)	저랭크 어댑터	매우 낮음	단일 GPU 특화

> 실습에서는 RunPod의 단일 GPU 인스턴스 기준이므로, **LoRA/QLoRA**가 현실적인 기본 선택지

4. 학습 방식: LoRA / QLoRA

- Full Fine-tuning의 한계
- LoRA 원리
- QLoRA: 양자화 + LoRA
- GPU 자원과 학습 비용

Full Fine-tuning의 한계

- VLM은 파라미터 수가 수십억 개 → 전체를 학습하면 **가중치**, **그래디언트**, **옵티마이저 상태**를 모두 GPU 메모리에 올려야 함
- 예: 7B 모델 full fine-tuning은 최소 수십 GB VRAM 필요 → 개인/소규모 팀에는 부담
- 대부분의 실전 fine-tuning은 **일부 파라미터만 학습**하는 PEFT(Parameter-Efficient Fine-Tuning) 방식을 사용

LoRA(Low-Rank Adaptation) 원리

- 기존 가중치 행렬 `W`는 그대로 두고(freeze), 옆에 **작은 저랭크 행렬 A, B**를 추가로 학습
- 학습·저장해야 할 파라미터 수가 원본 대비 수백 분의 1 수준으로 감소

```
h = Wx + (B·A)x
```

- `W` : 원본 가중치 (동결)
- `A, B` : 저랭크(rank r) 행렬 — 이 부분만 학습 (r은 보통 4~64)
- 학습 후 `B·A`만 저장하면 되므로 **결과물(체크포인트)도 매우 작음** (수십~수백 MB)

QLoRA: 양자화 + LoRA

- LoRA만으로도 학습 파라미터는 줄지만, **원본 모델 자체는 여전히 GPU에 올라가 있어야 함**
- QLoRA는 원본 가중치를 **4bit로 양자화**해 메모리에 올리고, 그 위에 LoRA(A, B)만 원래 정밀도로 학습

항목	LoRA	QLoRA
원본 가중치 정밀도	FP16/BF16	4bit 양자화
VRAM 사용량	중간	낮음
학습 속도	빠름	양자화 연산으로 약간 느려짐
적합한 상황	GPU 메모리 여유 있음	단일 저사양 GPU로 큰 모델 학습

> "더 큰 모델을 더 적은 GPU로" 학습 가능하게 하는 것이 QLoRA의 핵심 가치

GPU 자원과 학습 비용 (RunPod 기준)

모델 규모	권장 GPU(VRAM)	방식
~2B급 VLM	16GB급(예: T4 등)	LoRA
~7B급 VLM	24GB급(예: L4, RTX 4090 등)	QLoRA 권장
7B 초과	40GB급 이상 또는 다중 GPU	QLoRA + 자원 확장

- RunPod은 **시간 단위 과금**이므로, 학습 전에 데이터/코드/설정을 로컬에서 충분히 검증 후 GPU 인스턴스를 켜는 것이 비용 절감의 핵심
- 학습이 끝나면 **LoRA 가중치만 저장**하고 인스턴스는 즉시 종료

5. 평가: Fine-tuning 결과 확인

- Loss만으로는 부족한 이유
- 평가셋 구성 원칙
- 정량 평가 vs 정성 평가
- Fine-tuning 전후 비교
- 과적합·회귀(Regression) 체크

Loss만으로는 부족한 이유

- Training loss가 낮아졌다고 **실제 응답 품질이 좋아졌다는 보장은 없음**
- 데이터 양이 적을 때는 특정 패턴을 "이해"한 게 아니라 **암기**했을 가능성이 큼
- 최종적으로 확인해야 할 질문은 하나: **"실제 태스크에서 fine-tuning 전보다 더 나은 답을 내는가"**

> Loss 곡선은 학습이 "진행되고 있는지"를 보여줄 뿐, "쓸 만해졌는지"는 별도로 확인해야 함

평가셋 구성 원칙

- **Held-out**: 학습에 전혀 쓰지 않은 데이터로 평가 — train에 섞이면 암기 여부를 구분할 수 없음
- **분포 일치**: 실제 서비스에서 들어올 이미지/질문과 비슷한 비율로 구성(정상:불량 비율 등)
- **충분한 수**: 몇 장만으로는 우연히 맞은 것인지 실력인지 구분하기 어려움

항목	나쁜 예	좋은 예
데이터 출처	train과 같은 소스에서 임의로 몇 장 추출	처음부터 train/val을 분리해서 수집
케이스 구성	쉬운 케이스만 포함	애매한 케이스·엣지 케이스도 포함

정량 평가 vs 정성 평가

구분	정량 평가	정성 평가
방법	정확도·포맷 준수율 등 수치로 집계	사람이 답변을 직접 읽고 판단
장점	빠르고 반복 가능, 자동화 가능	수치로 못 잡는 미묘한 오류 포착
단점	채점 기준이 단순하면 실제 품질과 괴리 가능	느리고 평가자마다 기준이 다를 수 있음
실습 예	노트북 3의 `judge_answer` 키워드 매칭	정답과 답변을 나란히 놓고 직접 검수

> 정량 평가로 큰 흐름을 빠르게 확인하고, 애매한 사례는 정성 평가로 보완하는 것이 일반적

Fine-tuning 전후 비교

- 같은 평가셋에 **base 모델**과 **fine-tuned 모델**을 동일한 프롬프트로 돌려 결과를 나란히 비교
- 노트북 3(`Fine-tuning 전후 비교 평가.ipynb`)에서 이 방식을 그대로 구현 — base vs QLoRA 모델의 정확도를 val셋 기준으로 비교

```
base 모델 정확도:  00%
QLoRA 모델 정확도: 00%
```

> 수치 차이뿐 아니라, 어떤 유형의 사례에서 개선/악화됐는지 같이 확인해야 원인을 파악할 수 있음

과적합·회귀(Regression) 체크

- **과적합**: 학습 데이터와 매우 비슷한 사례만 잘 맞히고, 조금만 달라져도 틀리는지 확인
- **회귀(catastrophic forgetting)**: fine-tuning 전에는 잘하던 일반적인 태스크(예: 일반 이미지 캡셔닝)를 fine-tuning 후에 못하게 되지 않았는지 확인
- 도메인 평가셋 + 기존 일반 태스크 평가셋을 **함께** 돌려보는 것이 안전

> Fine-tuning은 특정 태스크를 잘하게 만드는 대신, 다른 능력을 희생시킬 수 있다는 트레이드오프를 항상 확인할 것

6. 경량 적응 방법: Prompt Tuning / VERA

- Prompt Tuning 개념
- VERA: Verbalized Learning 사례
- Prompt Tuning vs LoRA

Prompt Tuning 개념

- 모델 가중치는 전혀 건드리지 않고, **학습 가능한 프롬프트(soft prompt)** 벡터만 추가로 학습
- LoRA보다도 학습 파라미터가 훨씬 적음 — 그만큼 표현력도 제한적

구분	일반 프롬프트(하드 프롬프트)	Prompt Tuning(소프트 프롬프트)
형태	사람이 읽을 수 있는 자연어 텍스트	학습으로 최적화된 임베딩 벡터
설계 방법	사람이 직접 문구를 고민	데이터로부터 경사하강법으로 학습
해석 가능성	높음	낮음(벡터라 사람이 읽을 수 없음)
필요 자원	없음	소량의 학습 데이터·연산

VERA: Verbalized Learning 사례

VERA (Explainable Video Anomaly Detection via Verbalized Learning of VLMs)

- 기존 방식: 이상행동 탐지 규칙/기준을 사람이 프롬프트로 직접 서술
- VERA의 아이디어: **"좋은 판단 기준 자체"를 학습으로 도출**하고, 그 결과를 사람이 읽을 수 있는 자연어 규칙(guideline)으로 정리
- 학습된 규칙 문장을 프롬프트에 넣어 사용 → soft prompt와 달리 **결과물이 여전히 해석 가능한 텍스트**

> VERA는 soft prompt 학습이라기보다, **좋은 판단 기준을 언어화해서 추출하고 프롬프트에 활용하는 접근** — 4강의 프레임 기반 이상행동 탐지 실습과 이어지는 문제의식

> 실습: 노트북 4(`4. VERA 판단 기준 학습.ipynb`)에서 판단 기준을 자연어로 반복 개선하는 과정을 직접 구현

Prompt Tuning vs LoRA

구분	Prompt Tuning	LoRA
학습 파라미터	매우 적음	적음
표현력	제한적(단순 태스크 적응)	상대적으로 높음
다중 태스크 전환	프롬프트만 교체하면 됨(가벼움)	어댑터 파일 교체 필요
적합한 상황	페르소나·톤 고정, 경량 개인화	도메인 지식/패턴 자체를 바꿔야 할 때

7. 학습 대상 확장: Vision Adapter

- Vision Adapter 개념
- 적용 사례

Vision Adapter 개념

- Vision Encoder와 LLM 사이의 **Projector(연결부)를 확장·교체**하는 방식
- Vision Encoder, LLM 본체는 그대로 두고 **어댑터 모듈만 추가로 학습** → LoRA보다도 구조가 단순한 경량화 전략
- 새로운 이미지 도메인(예: 의료 영상, 특수 센서 이미지)에 기존 LLM의 언어 능력을 연결할 때 사용

> 이 구조를 대중화한 대표 사례가 **LLaVA**: Vision Encoder(CLIP) + Projector(MLP) + LLM(Vicuna)를 연결하고, 먼저 Projector만 학습해 이미지-언어를 정렬한 뒤 LLM과 함께 instruction tuning — 실습에서 쓴 Qwen3-VL의 `visual.merger`도 이름만 다를 뿐 같은 역할

적용 사례

- **도메인 전환**: 일반 이미지로 학습된 VLM을 의료 영상·위성 사진 등 새로운 시각 도메인에 연결
- **경량 개인화**: Vision Encoder·LLM은 그대로 두고 Projector만 재학습해 빠르게 특정 서비스에 맞춤
- LoRA와 함께 쓰는 경우도 많음 — Projector는 Vision Adapter로, LLM 쪽은 LoRA로 나눠 학습